



Throughput and Capacity

Early Access · MessageFoundry v0.3.2 · September 2026

How much traffic MessageFoundry carries, what we measured to establish it, and how to size your own deployment.

Summary

The published reference is **40 million message events per day**. It reserves **more than 20% of measured capacity**. The measurement used four engine processes with one database. This arrangement is **not yet a supported production topology**.

The measured ceiling was approximately **603 message events per second**, or about 52 million events per day. The published figure uses roughly three quarters of that rate. This is a point estimate from sustained holds, not a long-duration test.

Use these conditions when you read the figures:

- **A "message event" is one message in or one message out.** An interface that receives a message and delivers it to one destination produces two events.
- **At peak, nothing in the system was running out of capacity.** The database and the processors both had substantial headroom. The limit is a sequencing constraint, not exhausted hardware.
- **In real deployments the engine spends most of its time waiting for the systems it talks to.** Our measurements assume a partner that replies instantly. Yours will not.

1. What "message events per day" counts

This document counts **total message events**. Each inbound message is one event. Each outbound delivery is one event.

Shape	Events per message
One message in, one delivery out	2
One message in, four deliveries out	5

So 40 million events per day is roughly **20 million inbound messages** in a simple one-in-one-out feed, or fewer inbound messages where feeds fan out to several destinations.

Before you compare engines, identify the unit for each capacity figure. Total events, inbound messages, and deliveries are different units.

2. The measurement

Conditions

Topology	Four engine processes sharing one database
Store	Microsoft SQL Server on its own host, local NVMe storage
Fan-out tested	Both one-in-one-out and one-in-four-out
Duration	Sustained holds, not short bursts
Counted	Completed deliveries. The pipeline was confirmed drained. Offered messages were excluded.
Correctness	Zero loss. No messages remained stranded. Per-lane order was preserved. The pipeline drained fully.

Result

Approximately 600 to 605 message events per second, sustained.

The one-destination and four-destination tests reached the same total-event ceiling within measurement noise. Destination count and offered rate also changed between tests. Thus, the result does not isolate the effect of fan-out.

From ceiling to published figure

	Events/second	Per hour	Per day
Measured sustainable ceiling	~603	~2.17 M	~52.1 M
Published capacity (>20% reserve)	~463	~1.67 M	40 M

Publishing 40 million against a measured 52 million holds back about 23%.

Reserve provides capacity for bursts, retries, slower partners, and maintenance. At the measured ceiling, an increase in work can cause the backlog to grow.

3. Nothing was saturated at the ceiling

None of the resources listed below was exhausted at the measured ceiling.

Resource	At the ceiling	Exhausted?
Database CPU	~69% average, measured while deliberately overloaded 25% past the ceiling	No
Engine host CPU	Roughly half idle	No
Database commit capacity	~23,600–27,200 commits/second available against ~600 demanded	No — roughly 40× headroom
Connection pool	Never fully used. Connection acquisition waits averaged ~0.015 ms.	No
Worker thread pool	Queue essentially empty — zero for 84% of samples	No
Outbound delivery lanes	Around 90% idle	No

The evidence points to durable writes that must occur in sequence to preserve ordering and at-least-once delivery. The precise mechanism remains an inference.

Adding database processor cores did not increase the measured ceiling. A **28% reduction in committed transactions** changed sustained throughput by **less than one percent**, within measurement noise.

The store also operated below its measured commit ceiling. These results do not identify transaction reduction or faster log storage as effective ways to increase throughput. No effective change has been identified.

The proposed mechanism agrees with the measurements. It has not been isolated from all alternatives.

4. Why your throughput will differ: partner systems

Partner acknowledgement time can limit ordered delivery.

Ordered delivery is the default and is required for most clinical interfaces. The engine permits one message in flight per destination:

1. Send message N to the partner.
2. **Wait** while the partner receives it, processes it, usually writes it to its own database, and acknowledges it.
3. Only then send message $N+1$.

The wait includes network travel and partner processing. The next message cannot use that ordered stream until the wait ends.

Approximately:

messages/second $\approx 1000 \div (\text{engine's fixed per-message cost} + \text{partner round-trip})$, in milliseconds

Using the reference serial-path cost, a 50 ms acknowledgement gives about 15 messages per second. At 250 ms, the rate is about 4 messages per second. These are estimates for one ordered interface.

Every measurement used an **instantly acknowledging partner**. This isolates the engine contribution. The figures are reference ceilings, not forecasts for a deployment.

What you can do about a slow partner

- **Relax ordering where it is safe.** Feeds that tolerate out-of-order delivery let the engine keep many messages in flight at once, hiding the partner's round-trip behind concurrency.
- **Open multiple connections.** Each destination connection is its own ordered lane.
- **Split the feed at the source** — by facility, service line, or region — so aggregate volume is not gated by one serial lane.
- **Ask your partner about their acknowledgement time.** It is usually the cheapest thing to improve and the term that matters most.

5. Sizing a deployment

1. **Start from total events, not messages.** Multiply expected inbound volume by (1 + destinations per message).
2. **Size to the busy hour, not the daily average.** Healthcare traffic is not flat: volume is light overnight, climbs through the morning, and peaks during clinical hours. Use your own feed's busiest hour divided by its daily average — your integration team can measure this from existing interface logs, and it is worth doing rather than assuming.
3. **Apply your partners' acknowledgement times** (§4). For ordered feeds this is usually the largest single reduction.
4. **Scale by adding interfaces, not by making one interface faster.** A strictly ordered interface is a single serial lane with a bounded rate — that is the physics of guaranteed ordering, not a product limitation.

Interfaces share a database and an engine host. Their measured combined rate can be lower than the sum of their individual ceilings. Use a measured concurrent rate for sizing. Treat the sum as an upper bound.

6. Scope and limits

Keep these limits with each measurement:

- **Synthetic data, laboratory conditions.** All measurements use generated HL7 messages on dedicated test infrastructure. Nothing here was measured against a live clinical system.
- **Instant-acknowledging partner.** See §4. This is the largest single reason a production number will be lower.
- **Message size.** Measurements used compact messages typical of ADT traffic. Feeds carrying large embedded documents cost more per message.
- **Pass-through processing.** Heavy transformation, strict validation, and live enrichment lookups all reduce throughput, and are the largest hardware-independent factors.

- **Database state.** The measurement ran against a store already holding several million rows, which is representative of a system in service rather than a freshly initialised one.
- **Not externally audited.** These are our own measurements, reported with their conditions so you can judge how they map onto your environment.
- **No head-to-head comparisons.** We publish no competitive benchmark. Throughput figures are hardware- and workload-dependent, and a comparison run by one vendor against another is not evidence we would ask you to trust from anyone else.

Use throughput figures as references for tests in your environment. They do not guarantee deployment capacity.

Four questions to ask of any throughput claim

Ours or anyone's:

1. **What unit?** Total events, inbound messages, or deliveries?
2. **Ordered or unordered delivery?**
3. **How fast did the receiving system acknowledge?**
4. **What is the peak-hour rate behind that daily average?**

Capacity comparisons require the unit, ordering mode, partner response time, and peak-hour rate.

© 2026 **MessageFoundry, LLC** · MessageFoundry is licensed under **AGPL-3.0-or-later** · Document generated September 2026 for MessageFoundry v0.3.2.

MessageFoundry and the MessageFoundry word mark are trademarks of MessageFoundry, LLC. This document describes an **Early Access** release; check pypi.org/project/messagefoundry for the current version. Verify specifics against your own deployment and the engine's reference documentation.